

Stackly Capital

Investment & Wealth Advisory Enterprise System

COMPREHENSIVE TECHNICAL ARCHITECTURE SPECIFICATION

Introduction

Welcome to the technical architectural documentation for **Stackly Capital**, a state-of-the-art wealth management and investment advisory platform. In today's digital era, financial intelligence tools must combine rigorous mathematical precision with smooth, intuitive, and highly responsive user interfaces. Stackly Capital is built to fulfill these requirements by serving as a unified gateway for private clients, professional financial advisors, and system administrators.

Document Scope & Goals:

This specification documents the frontend design choices, mathematical algorithms, client-side data storage flows, and interactive views used in the Stackly Capital system. The core design philosophy centers on maximizing performance through clean, lightweight client-side execution, reducing the need for heavy external frameworks and keeping calculators and graphing engines responsive.

Document Navigation Guide:

- **System Architecture:** Explains the high-level design philosophy and technical stack decisions.
- **CSS Style & Design System:** Maps the typography, visual properties, color palettes, and glassmorphism cards.
- **Dynamic JavaScript Framework:** Outlines loader states, scroll animations, and interactive event triggers.
- **Calculators & Custom Charts:** Documents the mathematics of SIP/Lump Sum projections and canvas-based graph rendering.
- **Dashboard Integrations:** Breaks down individual components for Client, Advisor, and Admin portals.

Executive Summary & System Architecture

Stackly Capital is an enterprise-grade investment and wealth advisory platform. It bridges the gap between human-led wealth management and algorithmic analytics. Unlike typical modern SaaS applications which compile heavily to React, Vue, or Angular, Stackly Capital has been built strictly around native browser technologies (HTML5, Vanilla CSS3, ES6 JavaScript) complemented by the lightweight, responsive capabilities of the Bootstrap v5 framework.

Architectural Principles:

- **Zero-dependency Client Runtime:** The application eliminates excessive dependency package graphs. Rendering is done in-browser via fast vanilla selectors.
- **Lightweight Custom Charting:** Rather than using charting bundles exceeding 500KB, the system incorporates custom Canvas code to draw responsive charts locally.
- **Distributed Client-Side Execution:** Formulas for SIPs, taxes, and retirement options execute on the user's browser, eliminating API latency and reducing server compute charges.
- **Role Partitioning:** HTML pages separate the client, advisor, and administrator dashboard environments while maintaining a unified CSS visual language.

Data Security & Persistence Strategy:

The current deployment of Stackly Capital operates under a mock-persisted data architecture. User login credentials, portfolio returns, customizable settings (e.g. site theme), and notification state parameters are securely stored inside HTML5 LocalStorage. This keeps data sandboxed to the user's web browser, assuring immediate local data retrieval without network roundtrips.

Frontend CSS Design System & Theme Engine

Stackly Capital maintains a unified visual brand using CSS Custom Properties (variables) defined at the root selector. The styles are written inside the stylesheet. The palette features high-contrast typography, premium gold/emerald accents, and clean borders.

CSS Variables Definitions:

CSS Property Name	Default Value (Light Mode)	Function / Use Case
Font Display	'Sora', sans-serif	Used for prominent headers, hero banners, and cover pages.
Font Sans-Serif	'Plus Jakarta Sans', sans-serif	Core body text, descriptive blocks, and table cells.
Font Monospace	'JetBrains Mono', monospace	Displaying financial assets, interest rates, and numeric calculations.
Primary Brand Color	Emerald Green Hue (HSL 160)	Core brand green representing growth and financial health.
Background Base	Off-White Hue (HSL 150)	Main screen background for high readability.
Panel Background	Semi-Transparent White (Alpha 0.7)	Translucent container backgrounds for glassmorphism styles.
Text Primary	Dark Slate Hue (HSL 150)	Contrast color for primary text headers.
Accent Gradient	Linear Amber-Gold Gradient	Premium gold brand colors for highlight buttons.

Aesthetics & Glassmorphism: The layout uses translucent cards with backdrop filters and subtle borders. These parameters adjust automatically when dark mode is applied to the root document element.

UI Component Library & Layout Elements

The visual library defines custom component styles that overlay the core Bootstrap utilities. This architecture allows developers to construct modern dashboards quickly.

1. Glassmorphism Navigation Bar:

A sticky header bar utilizing translucent backdrops. As the user scrolls, JavaScript appends a scrolled class, which darkens the backdrop and applies a drop-shadow. This guarantees text legibility over any background.

2. Ripple-Effect Buttons:

All button interactions are enhanced using interactive JavaScript ripples. Upon clicking, standard coordinates are captured, a temporary ripple element is generated, and it scales up dynamically using CSS keyframe transitions.

3. Dashboard Cards:

Stat cards encapsulate financial metrics (e.g. portfolio valuations, cash yields). They incorporate hover state shifts, translating 4 pixels upward on the Y-axis and glowing slightly using amber outline shadows.

Reference Component Style Architecture:

The stylesheet declares specific layout definitions for navigation containers. It specifies position settings (sticky), stack index values (z-index of 1030), background configurations (nav-bg variable), backdrop filters (blur values), and transition times (0.3 seconds) to ensure smooth visual changes when scrolling.

Core Runtime JavaScript & Lifecycle Hook

The shared client logic resides in the main JavaScript file, operating under an immediately invoked function expression (IIFE) to prevent polluting the global namespace. It manages the app startup, transitions, and user settings.

Application Loading Lifecycle:

An animated overlay screen intercepts the viewport until dependencies are ready. The loading flow operates as follows:

Step	Trigger / Event	Action Taken by Runtime Script
1	HTML Parse Complete	Starts checking fonts and external assets.
2	DOMContentLoaded Event	Fades out loader opacity. Transition: 0.5 seconds.
3	Window Load Event	Completely removes loader DOM elements to clear memory.
4	Fallback Safety Timer	Forces loader removal after 600 milliseconds if assets are slow.

Scroll Reveal & Intersection Observer:

The system avoids constant scroll monitoring (which hurts frame rates). Instead, it sets up an intersection observer configured with a 12% intersection threshold. When elements containing the reveal class enter the viewport, the observer appends active class states, initiating a smooth CSS GPU-accelerated slide-up.

Interactive Canvas Charting Engine (Line & Bar)

The system renders portfolio valuations and advisory fees using HTML5 canvas shapes. The charting object inside the JavaScript files provides methods for drawing these graphs dynamically.

Line Chart Vector Rendering:

Renders continuous data arrays (such as portfolio valuation histories). The rendering process follows these steps:

1. Retrieves the canvas height and width, adjusting for the device's pixel ratio to prevent text blur.
2. Normalizes input data points against the maximum value in the array to compute scale factors.
3. Maps data indexes to the horizontal axis coordinate using normalized layout width fractions.
4. Translates data values to the vertical axis coordinate using scaled heights.
5. Draws the primary line, then applies a semi-transparent gradient fill below the curve.

Bar Chart Pillar Rendering:

Draws monthly cash inflows or outflow metrics. The function loops through data arrays, rendering custom rectangles using dual colors (e.g. deep blue and emerald green). It handles gridlines and numeric axis ticks automatically.

Line Chart Implementation Sequence:

The charting library establishes a canvas rendering context, clears coordinates, starts paths, sets line widths to 2.5 units, assigns strokes, and plots the points dynamically using normalized loop scales.

Charting Engine Part II: Donut & Portfolio Distribution

Donut charts are essential for showing portfolio diversification. The charting function calculates angular segments and draws circular sectors inside canvas wrappers.

Radial Math & Angle Interpolation:

The system reads percentages (such as 40% Stocks, 30% Bonds) from data attributes, translating them into radians:

- A full circle is defined mathematically as 2 multiplied by PI radians.
- Individual sector sizes are calculated by multiplying the percentage fraction by 2 and PI.
- The drawing starts from negative half PI (the top vertical position) and proceeds clockwise.

Donut Rendering Sequence:

Once the outer sectors are drawn, the canvas clears a inner central circle, overlaying it with the background theme color. This transforms the pie chart into a modern donut graphic, complete with middle-aligned text showing the total portfolio value.

Step	Canvas Action Command	Visual Objective
1	Draw Radial Segment Arc	Renders segment sector for specific asset category.
2	Set Fill Style Color	Colors segment using color indexes.
3	Draw Central Inner Arc	Clears the center to create the donut cutout.
4	Render Center Text	Draws portfolio asset text value in the middle.

Financial Calculators & SIP/Lump Sum Math

Stackly Capital includes client-facing financial planning tools. These calculators evaluate investment projections instantly using client-side formulas.

Systematic Investment Plan (SIP) Compound Math:

SIP allows investors to calculate future wealth growth from monthly contributions.

Formula Definition: The future value is calculated by taking the monthly contribution, multiplying it by the compounded rate factor minus one divided by the periodic rate, and finally multiplying by the interest adjustment factor for the initial deposit month.

Variables utilized: **P** is the Monthly Contribution, **i** is the monthly interest rate (annual interest rate / 12 / 100), and **n** is the total number of compounding periods (years × 12).

Lump Sum Compound Interest Math:

Lump Sum calculators compute growth for one-time principal deposits.

Formula Definition: The future value equals the principal amount multiplied by one plus the annual interest rate, raised to the power of the investment duration in years.

Calculators Parameter Constraints:

The calculator inputs are validated locally to ensure inputs are positive numbers. Minimum values are set to 100 units for contributions and rates are capped at 50% to prevent integer overflow during calculations.

Retirement Planner & Tax Estimation Logic

In addition to basic savings calculators, the platform features tools for retirement planning and tax estimation.

Inflation-Adjusted Retirement Planning:

The retirement planner projects savings by adjusting for inflation. It estimates future purchasing power by dividing the projected raw future value by one plus the inflation rate, raised to the power of the investment duration in years.

Bracket-Based Tax Estimator:

The tax calculator estimates liability based on progressive tax brackets:

Income Range (USD)	Tax Rate (%)	Incremental Tax Calculation
0 - \$11,000	10%	Ten percent of income
\$11,001 - \$44,725	12%	Base tax plus twelve percent of amount over bracket threshold
\$44,726 - \$95,375	22%	Base tax plus twenty-two percent of amount over bracket threshold
Above \$95,375	24%	Base tax plus twenty-four percent of amount over bracket threshold

Authentication Flow & LocalStorage Session Design

Stackly Capital manages user authentication entirely in the browser using LocalStorage and simple JavaScript redirect checks.

Registration Process:

Advisors can view an aggregated list of their clients. Selecting a client profile displays detailed analytics, risk tolerance levels, and pending asset allocations.

Login Validation & Session Initiation:

During login, the system validates inputs against stored credentials. If successful, it sets an authentication key and redirects the user to the client dashboard.

Profile Initial Generation:

To customize the user experience, the system parses the user's name to generate and display initials:

- **Single Name:** Uses the first two letters of the name.
- **Multiple Names:** Takes the first letter of each name.

Reference Authentication Logic:

The login handler retrieves stored profile keys. If no user is logged in, it redirects the browser to the login page. If authenticated, it updates profile name containers across the dashboard layout.

Client Portal Dashboard Architecture

The Client Portal is the primary user interface. It compiles asset allocations, recent transaction histories, and advisory updates into a single responsive view.

Sub-View Switching Architecture (SPA Simulation):

The system operates as a single-page application (SPA). Instead of loading new pages, navigation clicks trigger view-switching functions. This updates the URL hash, toggles active CSS classes on container elements, and triggers chart rendering.

Tab View Name	Core HTML Component	Display Metrics
Overview Panel	Total Balance, Daily Profit Cards	Primary portfolio balance and asset trend charts.
Portfolio Details	Asset Allocation Grid	Detailed allocation metrics and donut charts.
Goals Tracker	Retirement & Savings Progress Bars	Calculates and displays progress toward savings goals.
Advisor Session	Chat panel & Scheduler	Allows messaging with assigned advisors.
Documents Locker	PDF Uploads & Statements Table	Hosts statement records and transaction reports.

Real-time Message Simulation:

A simulated chat interface lets users message their advisor. Sent messages are dynamically appended to the layout with modern CSS message bubble styles, and a mock response is triggered after a brief delay.

Advisor Portal Dashboard Architecture

The Advisor Portal provides tools for advisors to manage client portfolios and coordinate investment planning.

Client Portfolio Management Matrix:

Advisors can view an aggregated list of their clients. Selecting a client profile displays detailed analytics, risk tolerance levels, and pending asset allocations.

Risk Tolerance Profile Adjustment:

Advisors can adjust a client's risk profile (Conservative, Moderate, Aggressive) in real-time. This adjustment re-calculates target asset allocations and updates client goals accordingly.

Feature Option	Interactive Elements	Trigger Logic Action
Client Selection	Client list sidebar buttons	Updates dashboards to display the selected client's data.
Allocation Editor	Segment sliders and input fields	Saves updated asset allocation matrices.
Meeting Scheduler	Calendar selection and time pickers	Generates advisory calendar links.

Administrator Portal Dashboard & Audit Console

The Admin Portal is designed for platform administrators to monitor system performance and manage user verification queues.

System Health Monitoring:

Displays real-time performance indicators, including server response times, active database connections, and memory usage.

User Verification & Compliance Queue:

Administrators can review and approve pending registrations, ensuring compliance with platform onboarding guidelines.

System Audit Log Stream Table:

Timestamp	Severity	Event Details
2026-07-24 11:58:12	INFO	User Sarah Jenkins completed KYC validation flow.
2026-07-24 11:59:03	WARNING	Advisor updated asset allocations for Client ID 98221.
2026-07-24 11:59:45	SUCCESS	LocalStorage cached sync state updated.

Codebase File Index & Client-Side Routing Reference

This chapter maps the location and routing of all static assets, styles, and scripts used throughout the platform.

File Path Location	Virtual HTTP Route	Functionality
Root Index File	Primary / Route	Primary home landing page.
About Page HTML	/about.html Route	Overview of company values and advisory board.
Services Page HTML	/services.html Route	Descriptions of wealth advisory services.
Plans Page HTML	/plans.html Route	Service subscription options.
Calculators Page HTML	/calculators.html Route	Financial planning calculators.
Client Dashboard HTML	/pages/client-dashboard.html Route	Client portal.
Advisor Dashboard HTML	/pages/advisor-dashboard.html Route	Advisor portal.
Admin Dashboard HTML	/pages/admin-dashboard.html Route	Administrative controls.

Performance Optimization & Verification Strategy

Stackly Capital targets high performance on various hardware configurations. The validation process ensures fast load times and clean component rendering.

Core Optimization Techniques:

- **Resource Compression:** Heavy image files are encoded in WebP format, reducing asset sizes by up to 60%.
- **Minimizing CSS Paint Latency:** Hardware acceleration is enabled for smooth transitions and card animations.
- **Font Subsetting:** Google Web Fonts are configured to load asynchronously, preventing blocking issues during page display.

Verification & Browser Testing Plan:

- **Mobile Responsiveness:** Verify layout performance using developer tools across various screen sizes (320px to 1920px).
- **Offline Persistence Testing:** Confirm that user preferences and inputs are saved correctly in local storage.
- **Calculation Auditing:** Cross-verify interest and wealth formulas against certified financial spreadsheets.

Security Guidelines, Compliance & Data Policies

This chapter outlines security considerations, data privacy practices, and compliance standards for the Stackly Capital platform.

Data Security & Storage:

Because user data is stored locally in the browser's LocalStorage, it is protected by the browser's origin-security policies. Developers should avoid saving sensitive data (such as raw passwords) in local storage, and should clear stored credentials when users log out.

Regulatory Compliance:

Financial platforms must adhere to guidelines regarding data disclosure and risk communication. The system displays standard disclaimers advising clients that historical performance does not guarantee future results.

Conclusion

In summary, **Stackly Capital** successfully delivers a modern, high-performance wealth management platform by utilizing clean client-side coding paradigms. The combination of flexible HTML5 components, customizable CSS variables, and a tailored JavaScript execution layer guarantees a fast, interactive experience for users. By hosting computations and layouts on the client side, the project achieves near-zero latency while remaining easy to maintain and expand.

Architectural Advantages:

- **Resource Efficiency:** Lightweight charting libraries and direct calculations reduce hosting costs.
- **Scalability Ready:** The organized directory layout and component architecture make it simple to integrate backend databases or secure APIs when the platform scales up.
- **User-Centric Design:** Responsive grids and smooth interface themes ensure a premium experience across desktops, tablets, and smartphones.